

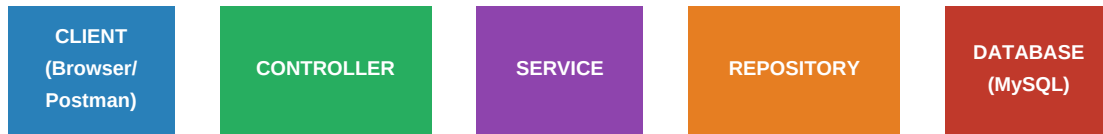
Daftar Isi

#	Bagian	Halaman
1	Gambaran Besar Teknologi	2
2	Java OOP Lengkap	3
3	Struktur Project Spring Boot	6
4	REST API & HTTP Status Code	7
5	Controller CRUD Lengkap	8
6	Database MySQL & JPA/ORM	9
7	Service & DTO	11
8	Exception Handling	12
9	File Transfer (Upload & Download)	13
10	Security (Spring Security + JWT)	14
11	Audit Trail	15
12	Testing dengan JUnit	16
13	Pagination & Sorting	17
14	Studi Kasus Step-by-Step	18
15	Cheat Sheet Anotasi & Tips	19
16	Checklist Siap Pelatihan	20

■ Dokumen ini adalah **FULL EDITION** — mencakup crash course OOP, setup project, CRUD lengkap, cheat sheet, dan seluruh rangkuman materi pelatihan.

1. Gambaran Besar Teknologi

Alur aplikasi backend Spring Boot dari request masuk hingga data tersimpan:



Lapisan	Tanggung Jawab	Anotasi Utama
Controller	Menerima HTTP request, kembalikan response	@RestController, @GetMapping, ...
Service	Logika bisnis / business logic	@Service, @Transactional
Repository	Akses & manipulasi data ke database	@Repository, JpaRepository
Entity	Representasi tabel di database	@Entity, @Table, @Id
DTO	Objek data transfer request/response	@Data (Lombok) / plain class

2. Java OOP (Object Oriented Programming)

2.1 Class & Object

Analogi: Class = cetakan kue, Object = kue yang sudah jadi

```
public class User { String nama; int umur; } // Membuat object dari class User
User u = new User(); u.nama = "Naila"; u.umur = 21;
```

2.2 Constructor

Analogi: Constructor = formulir yang harus diisi saat mendaftar — dijalankan otomatis saat 'new'

```
public class User { String nama; int umur; // Constructor
public User(String nama, int umur) { this.nama = nama; // 'this' = milik class ini sendiri
this.umur = umur; } } // Pemakaian User
u = new User("Naila", 21); System.out.println(u.nama); // "Naila"
```

2.3 Encapsulation

Analogi: ATM — kamu tidak tahu cara kerjanya di dalam, tapi bisa pakai tombolnya lewat getter/setter

```
public class User { private String nama; // private = tidak bisa diakses langsung dari luar
private int umur; // Getter – untuk MEMBACA nilai
public String getNama() { return nama; }
public int getUmur() { return umur; } // Setter – untuk MENGUBAH nilai
public void setNama(String nama) { this.nama = nama; }
public void setUmur(int umur) { this.umur = umur; }
}
```

Dengan Lombok @Data, semua getter/setter dibuat OTOMATIS tanpa perlu nulis manual!

2.4 Inheritance

Analogi: Anak mewarisi sifat orang tua — gunakan keyword 'extends'

```
public class Hewan { public void makan() { System.out.println("Makan"); } } // Kucing MEWARISI
semua method dari Hewan
public class Kucing extends Hewan { public void mengeong() { System.out.println("Meong!"); } }
Kucing c = new Kucing(); c.makan(); // diwarisi dari Hewan
c.mengeong(); // milik Kucing sendiri
```

2.5 Polymorphism

Analogi: Tombol 'share' di HP — perilakunya beda di setiap app, tapi nama methodnya sama

```
public class Hewan { public void suara() { System.out.println("Suara hewan..."); } }
public class Kucing extends Hewan { @Override // menandai bahwa method ini menimpa parent
public void suara() { System.out.println("Meong"); } }
public class Anjing extends Hewan { @Override public void suara() { System.out.println("Woof"); } }
```

2.6 Interface

Analogi: Job description — siapapun yang implements wajib mengerjakan semua tugas yang tertulis

```
public interface ProductService { List getAll(); ProductDTO getById(Long id); ProductDTO
create(ProductDTO dto); void delete(Long id); } // Class yang implements WAJIB override semua
method
public class ProductServiceImpl implements ProductService { @Override public List
getAll() { ... } @Override public ProductDTO getById(Long id) { ... } // ... dan seterusnya }
```

Interface sering dipakai di Spring: buat interface Service dulu, baru buat ServiceImpl-nya.

3. Struktur Project Spring Boot

Folder / File	Fungsi
controller/	Menerima HTTP request dari client
service/	Logika bisnis (interface)
service/impl/	Implementasi service (ServiceImpl)
repository/	Query & akses database
entity/	Representasi tabel database (POJO + JPA)
dto/	Data Transfer Object — request & response body
config/	Konfigurasi (Security, CORS, Beans, dll)
exception/	Custom exception & global error handler
DemoApplication.java	Entry point — main class Spring Boot
application.properties	Konfigurasi datasource, port, JPA, dll

Naming convention: ProductController, ProductService, ProductServiceImpl, ProductRepository, ProductEntity, ProductDTO

Setup Project Baru — Step by Step

1	Buat project di IntelliJ — File → New Project → Spring Initializr. Pilih: Java 17, Maven, packaging Jar.
2	Pilih dependency — Spring Web, Spring Data JPA, MySQL Driver, Lombok, Validation, Spring Security
3	Buat database — Buka DBeaver → jalankan: CREATE DATABASE training_db;
4	Isi application.properties — Lihat konfigurasi di Bab 6.1
5	Buat struktur folder — Buat folder: controller, service, repository, entity, dto, exception
6	Jalankan & cek — Klik Run  . Kalau berhasil muncul: Started DemoApplication in X seconds

4. REST API & HTTP Status Code

HTTP Method	Endpoint Contoh	Fungsi	Anotasi Spring
GET	/products	Ambil semua data	@GetMapping
GET	/products/1	Ambil data by ID	@GetMapping("/{id}")
POST	/products	Tambah data baru	@PostMapping
PUT	/products/1	Update seluruh data	@PutMapping("/{id}")
PATCH	/products/1	Update sebagian data	@PatchMapping("/{id}")
DELETE	/products/1	Hapus data	@DeleteMapping("/{id}")

HTTP Status Code

Kode	Status	Kapan Digunakan
200	OK	Request sukses (GET, PUT, DELETE)
201	Created	Data baru berhasil dibuat (POST)
204	No Content	Sukses tapi tidak ada data dikembalikan
400	Bad Request	Data request tidak valid / format salah
401	Unauthorized	Belum login / token tidak ada
403	Forbidden	Sudah login tapi tidak punya izin
404	Not Found	Data / endpoint tidak ditemukan
500	Internal Server Error	Error/bug di sisi server

5. Controller — CRUD Lengkap

```
@RestController @RequestMapping("/products") public class ProductController { @Autowired
private ProductService service; @GetMapping public List getAll() { return service.getAll(); }
@GetMapping("/{id}") public ResponseEntity getById(@PathVariable Long id) { return
ResponseEntity.ok(service.getById(id)); } @PostMapping public ResponseEntity
create(@RequestBody ProductDTO dto) { return
ResponseEntity.status(201).body(service.create(dto)); } @PutMapping("/{id}") public
ResponseEntity update( @PathVariable Long id, @RequestBody ProductDTO dto) { return
ResponseEntity.ok(service.update(id, dto)); } @DeleteMapping("/{id}") public ResponseEntity
delete(@PathVariable Long id) { service.delete(id); return ResponseEntity.noContent().build();
} }
```

Anotasi	Fungsi
@RestController	Gabungan @Controller + @ResponseBody
@RequestMapping	Prefix path untuk semua method di class ini
@PathVariable	Ambil nilai dari URL path e.g. /products/{id}
@RequestBody	Ambil data dari body request (JSON)
@RequestParam	Ambil query string e.g. ?page=1&size=10
ResponseEntity<>	Bungkus response dengan status code yang bisa dikustomisasi

6. Database MySQL & JPA / ORM

6.1 application.properties

```
spring.datasource.url=jdbc:mysql://localhost:3306/training_db spring.datasource.username=root
spring.datasource.password= spring.jpa.hibernate.ddl-auto=update spring.jpa.show-sql=true
spring.jpa.properties.hibernate.format_sql=true
spring.jpa.database-platform=org.hibernate.dialect.MySQL8Dialect server.port=8080
```

6.2 Entity Lengkap dengan Audit Trail

```
@Entity @Table(name = "products") @Data @NoArgsConstructor @AllArgsConstructor public class
Product { @Id @GeneratedValue(strategy = GenerationType.IDENTITY) private Long id;
@Column(nullable = false) private String name; @Column(nullable = false) private Double price;
@CreationTimestamp // otomatis isi saat data dibuat private LocalDateTime createdAt;
@UpdateTimestamp // otomatis update saat data diubah private LocalDateTime updatedAt; private
String createdBy; }
```

@CreationTimestamp dan @UpdateTimestamp = cara mudah buat audit trail di Spring Boot!

6.3 Repository

```
@Repository public interface ProductRepository extends JpaRepository { // Method bawaan: save,
findAll, findById, deleteById, existsById, count // Custom query – Spring bisa generate
otomatis dari nama method List findByNameContainingIgnoreCase(String keyword); List
findByPriceLessThan(Double price); // Custom query JPQL @Query("SELECT p FROM Product p WHERE
p.price BETWEEN :min AND :max") List findByPriceRange(@Param("min") Double min, @Param("max")
Double max); }
```

Method JpaRepository	SQL Setara
save(entity)	INSERT or UPDATE
findAll()	SELECT * FROM table
findById(id)	SELECT * WHERE id = ?
deleteById(id)	DELETE WHERE id = ?
existsById(id)	SELECT COUNT(*) WHERE id = ?
count()	SELECT COUNT(*) FROM table

7. Service & DTO

7.1 DTO dengan Validasi

```
@Data @NoArgsConstructor @AllArgsConstructor public class ProductDTO { private Long id;
@NotBlank(message = "Nama produk tidak boleh kosong") private String name; @Positive(message =
"Harga harus lebih dari 0") private Double price; }
```

7.2 Service Implementation

```
@Service public class ProductServiceImpl implements ProductService { @Autowired private
ProductRepository repository; @Override public List getAll() { return
repository.findAll().stream() .map(this::toDTO) .collect(Collectors.toList()); } @Override
public ProductDTO getById(Long id) { Product p = repository.findById(id) .orElseThrow(() -> new
ResourceNotFoundException("Produk tidak ditemukan, id: " + id)); return toDTO(p); } @Override
public ProductDTO create(ProductDTO dto) { Product p = new Product(); p.setName(dto.getName());
p.setPrice(dto.getPrice()); return toDTO(repository.save(p)); } @Override public void
delete(Long id) { if (!repository.existsById(id)) throw new ResourceNotFoundException("Produk
tidak ditemukan, id: " + id); repository.deleteById(id); } private ProductDTO toDTO(Product p)
{ return new ProductDTO(p.getId(), p.getName(), p.getPrice()); } }
```


8. Exception Handling

8.1 Custom Exception

```
public class ResourceNotFoundException extends RuntimeException { public  
ResourceNotFoundException(String message) { super(message); } }
```

8.2 Global Exception Handler

```
@ControllerAdvice public class GlobalExceptionHandler {  
@ExceptionHandler(ResourceNotFoundException.class) public ResponseEntity> handleNotFound(  
ResourceNotFoundException ex) { Map error = new HashMap<>(); error.put("status", "404");  
error.put("message", ex.getMessage()); return ResponseEntity.status(404).body(error); }  
@ExceptionHandler(MethodArgumentNotValidException.class) public ResponseEntity>  
handleValidation( MethodArgumentNotValidException ex) { Map errors = new HashMap<>();  
ex.getBindingResult().getFieldErrors() .forEach(e -> errors.put(e.getField(),  
e.getDefaultMessage())); return ResponseEntity.status(400).body(errors); }  
@ExceptionHandler(Exception.class) public ResponseEntity> handleGeneral(Exception ex) { Map  
error = new HashMap<>(); error.put("status", "500"); error.put("message", "Internal server  
error"); return ResponseEntity.status(500).body(error); } }
```

@ControllerAdvice menangkap semua exception dari seluruh controller secara global!

9. File Transfer (Upload & Download)

9.1 Konfigurasi

```
spring.servlet.multipart.max-file-size=10MB spring.servlet.multipart.max-request-size=10MB
file.upload-dir=./uploads
```

9.2 Controller Upload & Download

```
@RestController @RequestMapping("/files") public class FileController {
    @Value("${file.upload-dir}") private String uploadDir; @PostMapping("/upload") public
    ResponseEntity upload(@RequestParam MultipartFile file) { try { String filename =
    System.currentTimeMillis() + "_" + file.getOriginalFilename(); Path path =
    Paths.get(uploadDir).resolve(filename); Files.copy(file.getInputStream(), path,
    StandardCopyOption.REPLACE_EXISTING); return ResponseEntity.ok("File berhasil diupload: " +
    filename); } catch (IOException e) { return ResponseEntity.status(500).body("Gagal upload
    file"); } } @GetMapping("/download/{filename}") public ResponseEntity download(@PathVariable
    String filename) { try { Path path = Paths.get(uploadDir).resolve(filename); Resource resource
    = new UrlResource(path.toUri()); if (!resource.exists()) throw new
    ResourceNotFoundException("File tidak ditemukan"); return ResponseEntity.ok()
    .header(HttpHeaders.CONTENT_DISPOSITION, "attachment; filename=\"" + filename + "\"")
    .body(resource); } catch (Exception e) { throw new ResourceNotFoundException("File tidak dapat
    diunduh"); } } }
```

Selalu validasi tipe file (cek ekstensi & MIME type) untuk keamanan!

10. Security (Spring Security + JWT)

Spring Security digunakan untuk autentikasi (siapa kamu?) dan otorisasi (kamu boleh akses apa?).

10.1 Konsep JWT

Bagian JWT	Isi	Contoh
Header	Tipe token & algoritma	{ alg: HS256, typ: JWT }
Payload	Data user (claims)	{ sub: user, role: ADMIN, exp: ... }
Signature	Verifikasi integritas	HMACSHA256(base64Header+'.'+base64Payload, secret)

10.2 Alur Autentikasi JWT

```
1. User POST /auth/login -> { username, password } 2. Server validasi -> buat JWT token ->
  kirim ke client 3. Client simpan token (localStorage / memory) 4. Setiap request berikutnya:
  kirim header: Authorization: Bearer 5. Server verifikasi token -> proses request
```

10.3 Security Config

```
@Configuration @EnableWebSecurity public class SecurityConfig { @Bean public
SecurityFilterChain filterChain(HttpSecurity http) throws Exception { http .csrf().disable()
.sessionManagement() .sessionCreationPolicy(SessionCreationPolicy.STATELESS) .and()
.authorizeHttpRequests() .requestMatchers("/auth/**").permitAll()
.requestMatchers("/admin/**").hasRole("ADMIN") .anyRequest().authenticated(); return
http.build(); } @Bean public PasswordEncoder passwordEncoder() { return new
BCryptPasswordEncoder(); } }
```

JANGAN PERNAH simpan password plaintext! Selalu gunakan BCryptPasswordEncoder.

11. Audit Trail

Audit trail = pencatatan otomatis siapa yang membuat/mengubah data dan kapan waktunya.

```
// 1. Aktifkan di main class @SpringBootApplication @EnableJpaAuditing(auditorAwareRef =  
"auditorProvider") public class DemoApplication { ... } // 2. Buat AuditorAware  
@Component("auditorProvider") public class AuditorAwareImpl implements AuditorAware {  
@Override public Optional getCurrentAuditor() { Authentication auth =  
SecurityContextHolder.getContext().getAuthentication(); if (auth == null ||  
!auth.isAuthenticated()) return Optional.of("system"); return Optional.of(auth.getName()); } }  
// 3. Tambahkan di Entity @Entity @EntityListeners(AuditingEntityListener.class) public class  
Product { @CreatedDate private LocalDateTime createdAt; @LastModifiedDate private  
LocalDateTime updatedAt; @CreatedBy private String createdBy; @LastModifiedBy private String  
updatedBy; }
```

Dengan setup ini, setiap save/update otomatis mencatat waktu dan user yang melakukan perubahan.

12. Testing dengan JUnit

12.1 Unit Test — Service

```
@ExtendWith(MockitoExtension.class) class ProductServiceTest { @Mock private ProductRepository repository; @InjectMocks private ProductServiceImpl service; @Test void testGetAll_ShouldReturnList() { List dummy = List.of( new Product(1L, "Keyboard", 350000.0, null, null, null), new Product(2L, "Mouse", 150000.0, null, null, null) ); when(repository.findAll()).thenReturn(dummy); List result = service.getAll(); assertEquals(2, result.size()); assertEquals("Keyboard", result.get(0).getName()); } @Test void testGetById_NotFound_ShouldThrowException() { when(repository.findById(99L)).thenReturn(Optional.empty()); assertThrows(ResourceNotFoundException.class, () -> service.getById(99L)); } }
```

12.2 Integration Test — Controller

```
@SpringBootTest @AutoConfigureMockMvc class ProductControllerTest { @Autowired private MockMvc mockMvc; @Test void testGetAllProducts() throws Exception { mockMvc.perform(get("/products")).andExpect(status().isOk()); } @Test void testCreateProduct() throws Exception { String json = "{\"name\":\"Keyboard\",\"price\":350000}"; mockMvc.perform(post("/products").contentType(MediaType.APPLICATION_JSON).content(json)).andExpect(status().isCreated()).andExpect(jsonPath("$.name").value("Keyboard")); } }
```

@Mock = objek tiruan | @InjectMocks = inject ke class yang dites | when() = definisikan perilaku mock

13. Pagination & Sorting

Pagination penting agar aplikasi tidak mengirim semua data sekaligus, yang bisa memperlambat performa.

```
// Controller @GetMapping public Page getAll( @RequestParam(defaultValue = "0") int page,
@RequestParam(defaultValue = "10") int size, @RequestParam(defaultValue = "id") String sortBy)
{ Pageable pageable = PageRequest.of(page, size, Sort.by(sortBy)); return
service.getAll(pageable); } // Service public Page getAll(Pageable pageable) { return
repository.findAll(pageable).map(this::toDTO); } // Contoh request: // GET
/products?page=0&size=5&sortBy=name // Contoh response: // { "content": [...],
"totalElements": 50, "totalPages": 10, "number": 0 }
```

14. Studi Kasus — CRUD Product Lengkap

STEP 1	Buka IntelliJ, New Project → Spring Initializr Java 17, Maven, Jar. Dependency: Spring Web, Spring Data JPA, MySQL Driver, Lombok, Validation
STEP 2	Buat database di DBeaver Jalankan: CREATE DATABASE training_db;
STEP 3	Isi application.properties Konfigurasi datasource & JPA (lihat Bab 6.1)
STEP 4	Buat Entity Product.java di folder entity/ dengan @Entity, @Id, @CreationTimestamp
STEP 5	Buat Repository ProductRepository.java extends JpaRepository
STEP 6	Buat DTO ProductDTO.java di folder dto/ dengan validasi @NotBlank, @Positive
STEP 7	Buat Service ProductService.java (interface) + ProductServiceImpl.java lengkap CRUD
STEP 8	Buat Controller ProductController.java dengan GET, POST, PUT, DELETE
STEP 9	Buat Exception Handler GlobalExceptionHandler.java di folder exception/
STEP 10	Jalankan & Test di Postman Klik Run ■ → buka Postman → test semua endpoint

15. Cheat Sheet Anotasi & Tips

Anotasi Spring Boot

Anotasi	Fungsi
@RestController	Class adalah REST controller
@RequestMapping	Prefix path di class
@GetMapping	Handle HTTP GET
@PostMapping	Handle HTTP POST
@PutMapping	Handle HTTP PUT
@DeleteMapping	Handle HTTP DELETE
@PathVariable	Ambil dari URL /{id}
@RequestBody	Ambil dari JSON body
@RequestParam	Ambil query string ?k=v
@Autowired	Inject dependency otomatis
@Service	Tandai class sebagai service
@Repository	Tandai class sebagai repo
@Entity	Class = tabel database
@Id	Primary key
@GeneratedValue	Auto increment
@Column	Konfigurasi kolom
@Transactional	Wrap dalam DB transaction
@ControllerAdvice	Global exception handler
@ExceptionHandler	Handle exception tertentu

Lombok Annotations

Anotasi	Efek
@Data	Getter+Setter+toString+equals
@NoArgsConstructor	Constructor kosong
@AllArgsConstructor	Constructor semua field
@Builder	Builder pattern
@Slf4j	Logger otomatis
@RequiredArgsConstructor	Constructor final field

OOP Kilat

Konsep	Keyword / Cara
Class	public class Nama { }
Object	new NamaClass()
Constructor	method dengan nama = class
Encapsulation	private + getter/setter
Inheritance	extends NamaParent
Polymorphism	@Override method parent
Interface	implements NamaInterface
this.	Merujuk atribut class sendiri

16. Checklist Siap Pelatihan

- | |
|--|
| ☐ Java 17 terinstall (cek: java -version) |
| ☐ Maven berjalan (cek: mvn -v) |
| ☐ IntelliJ IDEA bisa dibuka dan berjalan normal |
| ☐ Laragon / XAMPP — MySQL berjalan |
| ☐ DBeaver bisa connect ke MySQL local |
| ☐ Postman terinstall dan bisa kirim request |
| ☐ Bisa membuat project Spring Boot baru via Spring Initializr |
| ☐ Bisa menjalankan aplikasi di localhost:8080 |
| ☐ Paham konsep GET POST PUT DELETE |
| ☐ Bisa membuat Entity sederhana dengan JPA |
| ☐ Paham konsep Class, Object, Inheritance di Java OOP |
| ☐ Sudah baca buku saku ini minimal sekali |

Kalau semua checklist terpenuhi — kamu siap ikut pelatihan dan mengerjakan studi kasus!